train_roberta_local

April 13, 2026

1 AGL — RoBERTa Fine-Tuning (Local)

Adaptive Guardrail Layer (AGL) — Binary Prompt Safety Classifier

Fine-tunes `roberta-base` on the AGL binary dataset (Benign vs Malicious). This notebook is designed for **local execution** (Apple Silicon MPS or CPU).

Data source: `data/processed/dataset_feature_engineered.csv`

[1]:
```
#   1. Setup
import os, sys

# Set working directory to project root (parent of notebooks/)
PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if os.path.basename(os.getcwd()) == 'notebooks':
    os.chdir(PROJECT_ROOT)
elif not os.path.exists('src/config.py'):
    if os.path.exists('../src/config.py'):
        os.chdir('..')

if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, os.getcwd())

print(f"Working directory: {os.getcwd()}")
assert os.path.exists('src/config.py'), "Not in project root - run from the␣
 ↪notebooks/ or project root directory"
```

```
Working directory: /home/admin/git/aai-590-group-8-capstone
```

[2]:
```
#   2. Check environment
import torch

print(f"Python: {sys.version}")
print(f"PyTorch: {torch.__version__}")

if hasattr(torch.backends, 'mps') and torch.backends.mps.is_available():
    DEVICE = "mps"
    print(f"Device: MPS (Apple Silicon)")
elif torch.cuda.is_available():
```

```
    DEVICE = "cuda"
    print(f"Device: CUDA - {torch.cuda.get_device_name(0)}")
else:
    DEVICE = "cpu"
    print(f"Device: CPU (training will be slow but it works)")

# Verify data exists
data_csv = 'data/processed/dataset_feature_engineered.csv'
assert os.path.exists(data_csv), f"Data not found at {data_csv}. Run the data␣
 ↪pipeline notebooks first."
print(f"\nData: {data_csv}  ")
```

```
Python: 3.14.2 (main, Jan 10 2026, 20:53:44) [GCC 15.2.1 20260103]
PyTorch: 2.10.0+cu128
Device: CUDA - NVIDIA GeForce RTX 5070 Ti

Data: data/processed/dataset_feature_engineered.csv
```

1.1 Step 3: Build Dataset

Load CSV, deduplicate, balance, and split into train/val/test parquets.

[3]:
```
#   3. Build dataset
from src.data.build_dataset import build_dataset

splits = build_dataset(csv_path='data/processed/dataset_feature_engineered.csv')

for name, df in splits.items():
    print(f"  {name}: {len(df)} samples")
```

```
============================================================
Building AGL dataset (binary classification)
============================================================

Loaded 104755 samples from data/processed/dataset_feature_engineered.csv
  Label distribution: {0: 55108, 1: 49647}
  Dedup removed 5665 duplicates

After dedup: 99090 samples

After balancing: 30000 samples
Label distribution:
label_name
Benign       15000
Malicious    15000
Saved train: 20999 samples ->
/home/admin/git/aai-590-group-8-capstone/data/processed/train.parquet
```

```
Saved val: 4500 samples ->
/home/admin/git/aai-590-group-8-capstone/data/processed/val.parquet
Saved test: 4501 samples ->
/home/admin/git/aai-590-group-8-capstone/data/processed/test.parquet
Saved metadata ->
/home/admin/git/aai-590-group-8-capstone/data/processed/dataset_metadata.json
  train: 20999 samples
  val: 4500 samples
  test: 4501 samples
```

[4]:
```python
#   3b. Verify dataset
import pandas as pd
from src.config import ID2LABEL

for split in ['train', 'val', 'test']:
    df = pd.read_parquet(f'data/processed/{split}.parquet')
    print(f"\n{split}: {len(df)} samples")
    label_counts = df['label'].value_counts()
    for label_id, count in label_counts.items():
        print(f"  {ID2LABEL[label_id]}: {count}")
```

```
train: 20999 samples
  Malicious: 10500
  Benign: 10499

val: 4500 samples
  Benign: 2250
  Malicious: 2250

test: 4501 samples
  Benign: 2251
  Malicious: 2250
```

1.2 Step 4: Fine-Tune RoBERTa (Transfer Learning)

Train the binary classifier with class-weighted cross-entropy, early stopping, and linear LR schedule.

Hyperparameters (from `src/config.py`): - LR: 2e-5, warmup: 10%, weight decay: 0.01 - Batch: 32, max epochs: 3, early stopping patience: 2 - Max sequence length: 128 - 2 classes: Benign (0), Malicious (1)

[5]:
```python
!pip show accelerate
```

```
Name: accelerate
Version: 1.13.0
Summary: Accelerate
Home-page: https://github.com/huggingface/accelerate
Author: The Hugging Face team
Author-email: transformers@huggingface.co
```

```
License: Apache
Location: /home/admin/git/aai-590-group-8-capstone/.venv/lib/python3.14/site-
packages
Requires: huggingface_hub, numpy, packaging, psutil, pyyaml, safetensors, torch
Required-by:
```

[6]:
```python
#   4. Train RoBERTa classifier
from src.training.train import train_classifier

best_path = train_classifier()
print(f"\nBest model saved to: {best_path}")
```

```
/home/admin/git/aai-590-group-8-capstone/.venv/lib/python3.14/site-
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
Map: 100%|      | 20999/20999 [00:00<00:00, 37720.27 examples/s]

Tokenized train: 20999 samples

Map: 100%|      | 4500/4500 [00:00<00:00, 31960.71 examples/s]

Tokenized val: 4500 samples

Map: 100%|      | 4501/4501 [00:00<00:00, 34349.39 examples/s]

Tokenized test: 4501 samples
Class weights: [1.0000476235831983, 0.9999523809523809]

Loading weights: 100%|      | 197/197 [00:00<00:00, 25234.48it/s]
RobertaForSequenceClassification LOAD REPORT from: roberta-base
Key                             | Status     |
--------------------------------+------------+-
lm_head.bias                    | UNEXPECTED |
lm_head.layer_norm.weight       | UNEXPECTED |
lm_head.dense.bias              | UNEXPECTED |
lm_head.layer_norm.bias         | UNEXPECTED |
roberta.embeddings.position_ids | UNEXPECTED |
lm_head.dense.weight            | UNEXPECTED |
classifier.dense.weight         | MISSING    |
classifier.dense.bias           | MISSING    |
classifier.out_proj.weight      | MISSING    |
classifier.out_proj.bias        | MISSING    |

Notes:
- UNEXPECTED        :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the
```

```
checkpoint. Consider training on your downstream task.
warmup_ratio is deprecated and will be removed in v5.2. Use `warmup_steps`
instead.

Starting training: 3 epochs, lr=2e-05
<IPython.core.display.HTML object>

[Epoch 1] val_loss=0.2520  macro_f1=0.8986  accuracy=0.8991
Writing model shards: 100%|     | 1/1 [00:00<00:00,  3.15it/s]

[Epoch 2] val_loss=0.2180  macro_f1=0.9124  accuracy=0.9124
Writing model shards: 100%|     | 1/1 [00:00<00:00,  3.16it/s]

[Epoch 3] val_loss=0.2651  macro_f1=0.9200  accuracy=0.9200
Writing model shards: 100%|     | 1/1 [00:00<00:00,  3.10it/s]
There were missing keys in the checkpoint model loaded:
['roberta.embeddings.LayerNorm.weight', 'roberta.embeddings.LayerNorm.bias',
'roberta.encoder.layer.0.attention.output.LayerNorm.weight',
'roberta.encoder.layer.0.attention.output.LayerNorm.bias',
'roberta.encoder.layer.0.output.LayerNorm.weight',
'roberta.encoder.layer.0.output.LayerNorm.bias',
'roberta.encoder.layer.1.attention.output.LayerNorm.weight',
'roberta.encoder.layer.1.attention.output.LayerNorm.bias',
'roberta.encoder.layer.1.output.LayerNorm.weight',
'roberta.encoder.layer.1.output.LayerNorm.bias',
'roberta.encoder.layer.2.attention.output.LayerNorm.weight',
'roberta.encoder.layer.2.attention.output.LayerNorm.bias',
'roberta.encoder.layer.2.output.LayerNorm.weight',
'roberta.encoder.layer.2.output.LayerNorm.bias',
'roberta.encoder.layer.3.attention.output.LayerNorm.weight',
'roberta.encoder.layer.3.attention.output.LayerNorm.bias',
'roberta.encoder.layer.3.output.LayerNorm.weight',
'roberta.encoder.layer.3.output.LayerNorm.bias',
'roberta.encoder.layer.4.attention.output.LayerNorm.weight',
'roberta.encoder.layer.4.attention.output.LayerNorm.bias',
'roberta.encoder.layer.4.output.LayerNorm.weight',
'roberta.encoder.layer.4.output.LayerNorm.bias',
'roberta.encoder.layer.5.attention.output.LayerNorm.weight',
'roberta.encoder.layer.5.attention.output.LayerNorm.bias',
'roberta.encoder.layer.5.output.LayerNorm.weight',
'roberta.encoder.layer.5.output.LayerNorm.bias',
'roberta.encoder.layer.6.attention.output.LayerNorm.weight',
'roberta.encoder.layer.6.attention.output.LayerNorm.bias',
'roberta.encoder.layer.6.output.LayerNorm.weight',
```

```
'roberta.encoder.layer.6.output.LayerNorm.bias',
'roberta.encoder.layer.7.attention.output.LayerNorm.weight',
'roberta.encoder.layer.7.attention.output.LayerNorm.bias',
'roberta.encoder.layer.7.output.LayerNorm.weight',
'roberta.encoder.layer.7.output.LayerNorm.bias',
'roberta.encoder.layer.8.attention.output.LayerNorm.weight',
'roberta.encoder.layer.8.attention.output.LayerNorm.bias',
'roberta.encoder.layer.8.output.LayerNorm.weight',
'roberta.encoder.layer.8.output.LayerNorm.bias',
'roberta.encoder.layer.9.attention.output.LayerNorm.weight',
'roberta.encoder.layer.9.attention.output.LayerNorm.bias',
'roberta.encoder.layer.9.output.LayerNorm.weight',
'roberta.encoder.layer.9.output.LayerNorm.bias',
'roberta.encoder.layer.10.attention.output.LayerNorm.weight',
'roberta.encoder.layer.10.attention.output.LayerNorm.bias',
'roberta.encoder.layer.10.output.LayerNorm.weight',
'roberta.encoder.layer.10.output.LayerNorm.bias',
'roberta.encoder.layer.11.attention.output.LayerNorm.weight',
'roberta.encoder.layer.11.attention.output.LayerNorm.bias',
'roberta.encoder.layer.11.output.LayerNorm.weight',
'roberta.encoder.layer.11.output.LayerNorm.bias'].
There were unexpected keys in the checkpoint model loaded:
['roberta.embeddings.LayerNorm.beta', 'roberta.embeddings.LayerNorm.gamma',
'roberta.encoder.layer.0.attention.output.LayerNorm.beta',
'roberta.encoder.layer.0.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.0.output.LayerNorm.beta',
'roberta.encoder.layer.0.output.LayerNorm.gamma',
'roberta.encoder.layer.1.attention.output.LayerNorm.beta',
'roberta.encoder.layer.1.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.1.output.LayerNorm.beta',
'roberta.encoder.layer.1.output.LayerNorm.gamma',
'roberta.encoder.layer.2.attention.output.LayerNorm.beta',
'roberta.encoder.layer.2.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.2.output.LayerNorm.beta',
'roberta.encoder.layer.2.output.LayerNorm.gamma',
'roberta.encoder.layer.3.attention.output.LayerNorm.beta',
'roberta.encoder.layer.3.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.3.output.LayerNorm.beta',
'roberta.encoder.layer.3.output.LayerNorm.gamma',
'roberta.encoder.layer.4.attention.output.LayerNorm.beta',
'roberta.encoder.layer.4.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.4.output.LayerNorm.beta',
'roberta.encoder.layer.4.output.LayerNorm.gamma',
'roberta.encoder.layer.5.attention.output.LayerNorm.beta',
'roberta.encoder.layer.5.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.5.output.LayerNorm.beta',
'roberta.encoder.layer.5.output.LayerNorm.gamma',
'roberta.encoder.layer.6.attention.output.LayerNorm.beta',
```

```
'roberta.encoder.layer.6.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.6.output.LayerNorm.beta',
'roberta.encoder.layer.6.output.LayerNorm.gamma',
'roberta.encoder.layer.7.attention.output.LayerNorm.beta',
'roberta.encoder.layer.7.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.7.output.LayerNorm.beta',
'roberta.encoder.layer.7.output.LayerNorm.gamma',
'roberta.encoder.layer.8.attention.output.LayerNorm.beta',
'roberta.encoder.layer.8.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.8.output.LayerNorm.beta',
'roberta.encoder.layer.8.output.LayerNorm.gamma',
'roberta.encoder.layer.9.attention.output.LayerNorm.beta',
'roberta.encoder.layer.9.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.9.output.LayerNorm.beta',
'roberta.encoder.layer.9.output.LayerNorm.gamma',
'roberta.encoder.layer.10.attention.output.LayerNorm.beta',
'roberta.encoder.layer.10.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.10.output.LayerNorm.beta',
'roberta.encoder.layer.10.output.LayerNorm.gamma',
'roberta.encoder.layer.11.attention.output.LayerNorm.beta',
'roberta.encoder.layer.11.attention.output.LayerNorm.gamma',
'roberta.encoder.layer.11.output.LayerNorm.beta',
'roberta.encoder.layer.11.output.LayerNorm.gamma'].
Writing model shards: 100%|      | 1/1 [00:00<00:00,  3.61it/s]


Best model saved →
/home/admin/git/aai-590-group-8-capstone/models/classifier/best

Best model saved to:
/home/admin/git/aai-590-group-8-capstone/models/classifier/best
```

1.3 Step 5: Fit Anomaly Detector (Optional)

Extract [CLS] embeddings from the fine-tuned model and fit the Mahalanobis OOD detector. Threshold is calibrated on the validation set (95% in-distribution recall).

[7]:
```python
#   5. Fit Mahalanobis anomaly detector (optional)
from src.training.train import train_anomaly

anomaly_path = train_anomaly()
print(f"\nAnomaly detector saved to: {anomaly_path}")
```

```
Loading weights: 100%|      | 201/201 [00:00<00:00, 26791.72it/s]
Map: 100%|      | 20999/20999 [00:00<00:00, 39233.57 examples/s]

Tokenized train: 20999 samples
```

```
Map: 100%|      | 4500/4500 [00:00<00:00, 30832.86 examples/s]

Tokenized val: 4500 samples

Map: 100%|      | 4501/4501 [00:00<00:00, 34497.77 examples/s]

Tokenized test: 4501 samples
Extracting [CLS] embeddings from training set…
Extracting [CLS] embeddings from validation set…
Fitting Mahalanobis detector (PCA → 100 dims)…
[anomaly] Calibrated threshold: 252.2693 (95% ID recall)
[anomaly] Saved detector →
/home/admin/git/aai-590-group-8-capstone/models/anomaly

Anomaly detector saved to:
/home/admin/git/aai-590-group-8-capstone/models/anomaly
```

1.4 Step 6: Evaluate

Run the full evaluation suite: keyword baseline, TF-IDF/SVM baseline, RoBERTa classifier.

[8]:
```python
#   6. Run evaluation
import numpy as np
import pandas as pd
from src.config import MODELS_DIR, PROCESSED_DIR, RESULTS_DIR
from src.evaluation.metrics import evaluate_predictions, save_results
from src.evaluation.baselines import keyword_blocklist_baseline,␣
 ↪tfidf_svm_baseline

# Load test data
train_df = pd.read_parquet(PROCESSED_DIR / 'train.parquet')
test_df = pd.read_parquet(PROCESSED_DIR / 'test.parquet')
y_true = test_df['label'].values

print("=" * 60)
print("Running evaluation suite")
print("=" * 60)

all_results = {}

# Baseline 1: Keyword blocklist
print("\n[1/3] Keyword blocklist baseline...")
kw_preds = keyword_blocklist_baseline(test_df)
kw_results = evaluate_predictions(y_true, kw_preds)
all_results['keyword_blocklist'] = kw_results
print(f"  Macro-F1: {kw_results['macro_f1']:.4f}")

# Baseline 2: TF-IDF + SVM
print("\n[2/3] TF-IDF + LinearSVM baseline...")
```

```
svm_preds, _ = tfidf_svm_baseline(train_df, test_df)
svm_results = evaluate_predictions(y_true, svm_preds)
all_results['tfidf_svm'] = svm_results
print(f"  Macro-F1: {svm_results['macro_f1']:.4f}")

# RoBERTa
checkpoint = MODELS_DIR / 'classifier' / 'best'
if checkpoint.exists():
    print("\n[3/3] RoBERTa classifier...")
    from src.models.agl_pipeline import AGLPipeline
    pipeline = AGLPipeline.from_checkpoint(checkpoint)
    roberta_preds = np.array([pipeline.predict(t).label_id for t in␣
 ↪test_df['text']])
    roberta_results = evaluate_predictions(y_true, roberta_preds)
    all_results['roberta'] = roberta_results
    print(f"  Macro-F1: {roberta_results['macro_f1']:.4f}")
else:
    print(f"\n[3/3] Skipping RoBERTa - no checkpoint at {checkpoint}")

save_results(all_results, 'evaluation_results', RESULTS_DIR)

print("\n" + "=" * 60)
print("Summary - Macro-F1 Scores:")
print("=" * 60)
for method, res in all_results.items():
    print(f"  {method:25s} {res['macro_f1']:.4f}")
```

```
============================================================
Running evaluation suite
============================================================

[1/3] Keyword blocklist baseline…
  Macro-F1: 0.3626

[2/3] TF-IDF + LinearSVM baseline…
  Macro-F1: 0.8553

[3/3] RoBERTa classifier…
Loading weights: 100%|      | 201/201 [00:00<00:00, 27275.39it/s]
  Macro-F1: 0.9180
Saved results → 
/home/admin/git/aai-590-group-8-capstone/results/evaluation_results.json

============================================================
Summary - Macro-F1 Scores:
============================================================
  keyword_blocklist         0.3626
```

```
tfidf_svm                 0.8553
roberta                   0.9180
```

1.5 Step 7: Visualizations

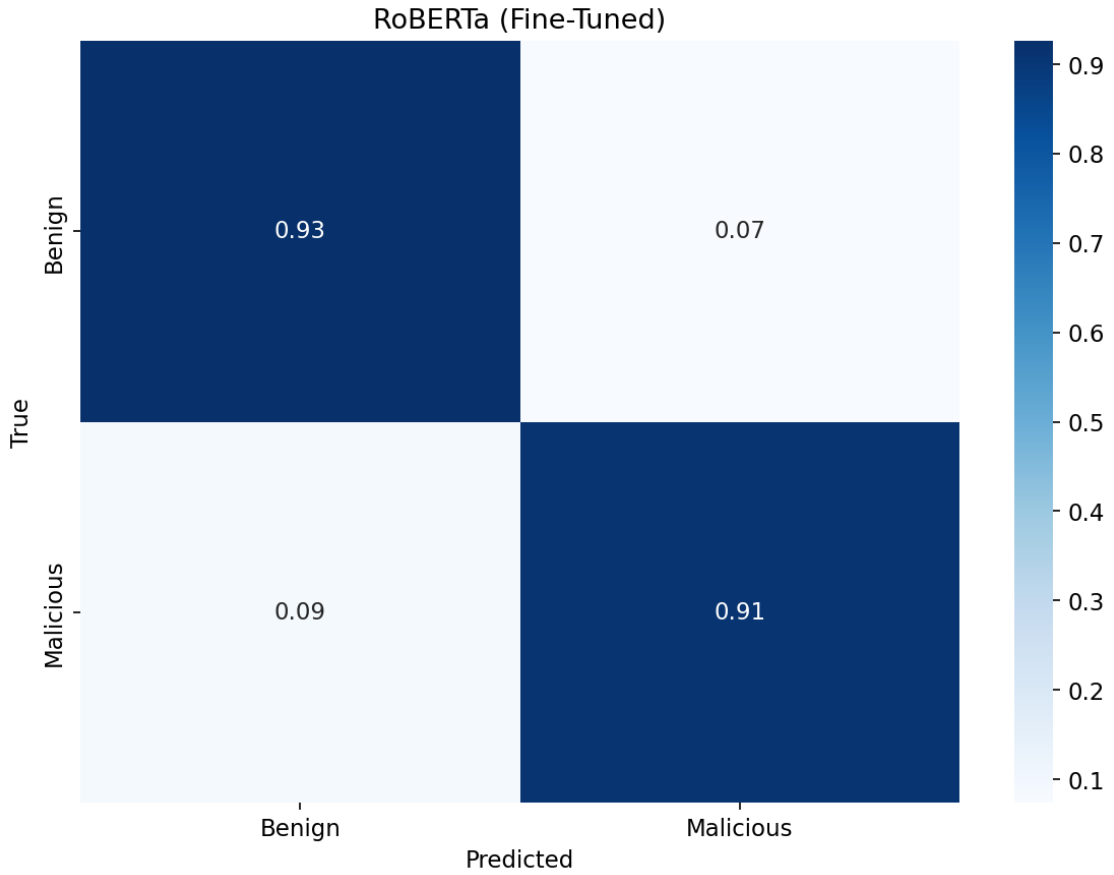
Generate figures: confusion matrix, F1 comparison, ROC curve, latency distribution.

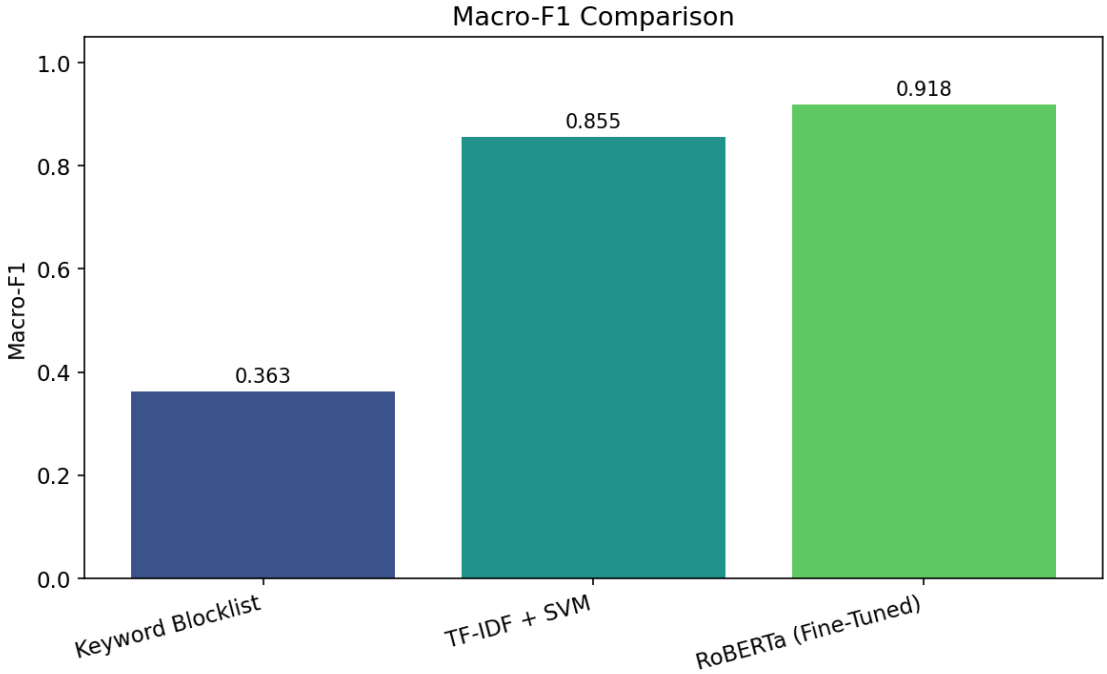
[9]:
```
#   7. Generate visualizations
import numpy as np
import pandas as pd
import torch
from src.config import RESULTS_DIR, FIGURES_DIR, PROCESSED_DIR, MODELS_DIR,␣
 ↪MAX_SEQ_LENGTH
from src.evaluation.visualizations import (
    plot_confusion_matrix, plot_f1_comparison, plot_roc_curves
)
from src.evaluation.metrics import evaluate_predictions, benchmark_latency,␣
 ↪save_results
from src.evaluation.baselines import keyword_blocklist_baseline,␣
 ↪tfidf_svm_baseline
from src.models.agl_pipeline import AGLPipeline
from transformers import AutoTokenizer

FIGURES_DIR.mkdir(parents=True, exist_ok=True)

# Load test data
test_df = pd.read_parquet(PROCESSED_DIR / 'test.parquet')
train_df = pd.read_parquet(PROCESSED_DIR / 'train.parquet')
y_true = test_df['label'].values

# Baselines
kw_preds = keyword_blocklist_baseline(test_df)
svm_preds, _ = tfidf_svm_baseline(train_df, test_df)

# RoBERTa predictions + probabilities (for ROC-AUC)
checkpoint = MODELS_DIR / 'classifier' / 'best'
pipeline = AGLPipeline.from_checkpoint(checkpoint)
tokenizer = AutoTokenizer.from_pretrained(str(checkpoint),␣
 ↪local_files_only=True)

roberta_preds = []
roberta_probs = []
for text in test_df['text']:
    pred = pipeline.predict(text)
    roberta_preds.append(pred.label_id)
    inputs = tokenizer(text, truncation=True, padding='max_length',
                       max_length=MAX_SEQ_LENGTH, return_tensors='pt')
```

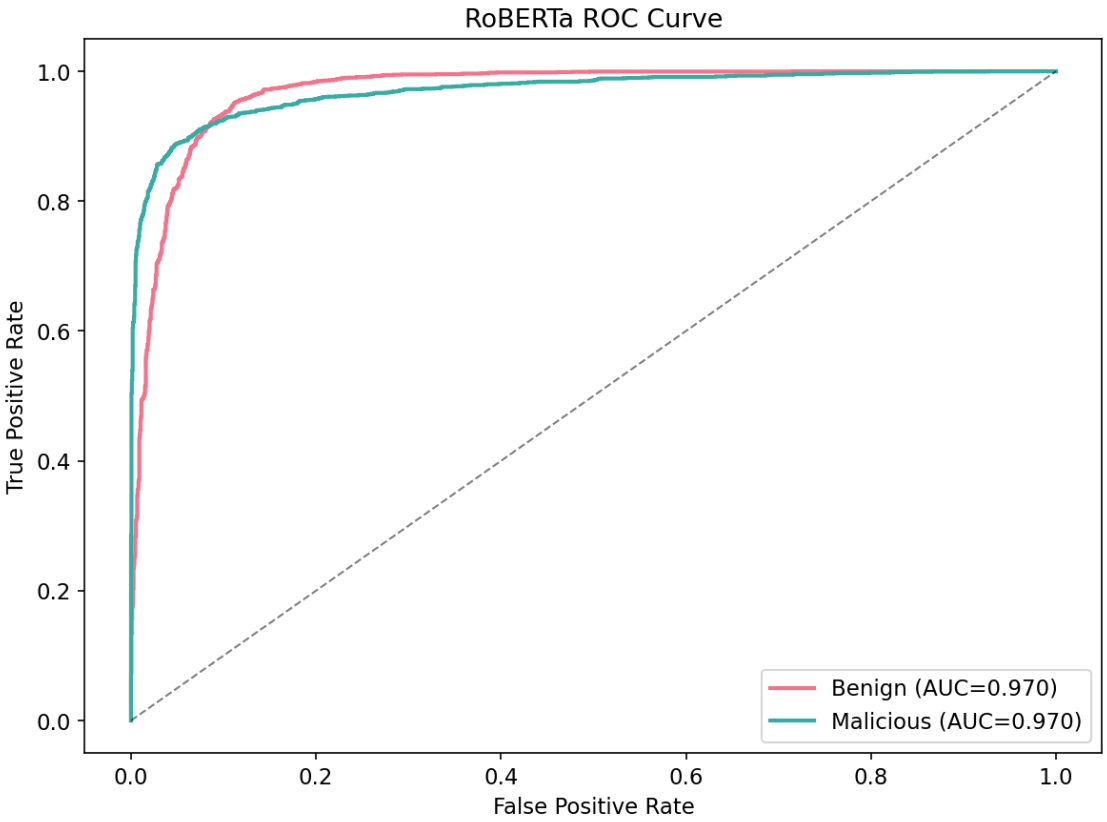
```
    inputs = {k: v.to(pipeline.device) for k, v in inputs.items()}
    with torch.no_grad():
        logits = pipeline.model(**inputs).logits
        probs = torch.softmax(logits, dim=-1).cpu().numpy()[0]
    roberta_probs.append(probs)

roberta_preds = np.array(roberta_preds)
roberta_probs = np.array(roberta_probs)

#   Confusion matrix
plot_confusion_matrix(y_true, roberta_preds, title='RoBERTa (Fine-Tuned)',
                      save_path=FIGURES_DIR / 'confusion_matrix_roberta.png')

#   F1 comparison
from sklearn.metrics import f1_score
f1_results = {
    'Keyword Blocklist': f1_score(y_true, kw_preds, average='macro',␣
 ↪zero_division=0),
    'TF-IDF + SVM': f1_score(y_true, svm_preds, average='macro',␣
 ↪zero_division=0),
    'RoBERTa (Fine-Tuned)': f1_score(y_true, roberta_preds, average='macro',␣
 ↪zero_division=0),
}
plot_f1_comparison(f1_results, save_path=FIGURES_DIR / 'f1_comparison.png')

#   ROC Curve
plot_roc_curves(y_true, roberta_probs, title='RoBERTa ROC Curve',
                save_path=FIGURES_DIR / 'roc_curve_roberta.png')

#   Full evaluation with AUC
results = evaluate_predictions(y_true, roberta_preds, y_prob=roberta_probs)
print(f"\nRoBERTa Results:")
print(f"  Macro-F1:  {results['macro_f1']:.4f}")
print(f"  ROC-AUC:   {results.get('roc_auc', 'N/A')}")
print(f"  PR-AUC:    {results.get('pr_auc', 'N/A')}")

#   Latency
latency = benchmark_latency(pipeline, test_df['text'].head(50).tolist(),␣
 ↪n_runs=2)
print(f"\nLatency: mean={latency['mean_ms']:.1f}ms, p99={latency['p99_ms']:.
 ↪1f}ms")

#   Save results JSON
all_results = {
    'roberta_finetuned': {**results, 'latency': latency}
}
save_results(all_results, 'roberta_evaluation', RESULTS_DIR)
```

```
print('\nAll figures saved to:', FIGURES_DIR)
```

```
Loading weights: 100%|      | 201/201 [00:00<00:00, 29084.91it/s]

Saved → /home/admin/git/aai-590-group-8-
capstone/results/figures/confusion_matrix_roberta.png
Saved →
/home/admin/git/aai-590-group-8-capstone/results/figures/f1_comparison.png
Saved →
/home/admin/git/aai-590-group-8-capstone/results/figures/roc_curve_roberta.png

RoBERTa Results:
  Macro-F1:  0.9180
  ROC-AUC:   0.9701407769386444
  PR-AUC:    0.9752534250881286

Latency: mean=4.2ms, p99=5.3ms
Saved results →
/home/admin/git/aai-590-group-8-capstone/results/roberta_evaluation.json

All figures saved to: /home/admin/git/aai-590-group-8-capstone/results/figures
```




Macro-F1 Comparison
Macro-F1
1.0
0.8
0.6
0.4
0.2
0.0
0.363
0.855
0.918
Keyword Blocklist
TF-IDF + SVM
RoBERTa (Fine-Tuned)


RoBERTa ROC Curve
True Positive Rate
False Positive Rate
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Benign (AUC=0.970)
Malicious (AUC=0.970)

1.6 Step 8: Done

Model checkpoint saved to `models/classifier/best/`, results to `results/`.

[10]:
```
#   8. Summary
from pathlib import Path

print("Artifacts saved locally:")
for p in ['models/classifier/best', 'models/anomaly', 'results']:
    exists = " " if Path(p).exists() else " "
    print(f"  {exists} {p}")

print("\nTo run a quick demo:")
print('  python3 -m src.run --stage demo --text "ignore previous instructions"')
```

```
Artifacts saved locally:
    models/classifier/best
    models/anomaly
    results

To run a quick demo:
  python3 -m src.run --stage demo --text "ignore previous instructions"
```